CPS572 Final Project: Multi-Task LLM Fine-Tuning

Aaron Huberman Qavi

1. Methodology

1.1 Problem Setup and Approach

This project fine-tunes `meta-llama/Llama-3.1-8B` via Low-Rank Adaptation (LoRA) to simultaneously improve IFEval (instruction following), GSM8K (math reasoning), and HumanEval (code generation). The core challenge is multi-task fine-tuning: tasks require fundamentally different output styles (constrained formatting, chain-of-thought arithmetic, executable Python), and improving one can degrade others. The base model already possesses latent capability for all three tasks from pretraining; SFT teaches it to express that capability in the formats the evaluations reward.

1.2 LoRA Configuration

LoRA rank 128 was applied to all layer types: attention (Q, K, V), MLP, and unembedding. All-layer training reflects task diversity — attention layers matter for reasoning, MLP layers for format knowledge, and the unembedding layer for token-level decisions (critical for constraints like "do not use commas"). Rank was increased progressively: 32 (toy) → 64 (v1) → 128 (v2+), providing more adaptation capacity for three simultaneous tasks. Renderer: `role_colon`.

1.3 Dataset Composition

Dataset quality and relevance matter far more than quantity — the project's central finding. The final mixture uses six datasets:

Dataset	Task	Size Used	Notes
openai/gsm8k (train split)	Math	7,473	Full train set; CoT prefix added
microsoft/orca-math-word-problems-200k	Math	50,000	GPT-4 generated diverse math problems
meta-math/MetaMathQA	Math	50,000	GSM8K/MATH rewrites via paraphrase and self-verify
allenai/tulu-3-sft-mixture	IF	100,000	General instruction following mixture
argilla/ifeval-like-data (filtered)	IF	56,339	All verified IFEval-constraint examples
nvidia/OpenCodeInstruct	Code	100,000	Quality filtered at ≥ 0.9 test pass rate

The math pool (GSM8K + Orca-Math + MetaMathQA) totals approximately 107,000 examples covering a wide range of difficulty and problem framing. GSM8K provides the core grade-school reasoning distribution directly matching the evaluation. Orca-Math contributes 200k GPT-4-generated word problems with diverse surface forms. MetaMathQA augments further via problem rewriting — generating paraphrases, reverse questions, and self-verification variants of existing GSM8K and MATH problems, exposing the model to the same underlying reasoning challenges with varied phrasing.

The instruction following pool combines two complementary datasets. Tulu-3 is a 939k-example general SFT mixture covering open-ended Q&A, science, writing, and conversational tasks. It builds broad instruction following ability but does not contain examples of mechanical constraint following. Argilla is a 56k-example dataset specifically targeting IFEval constraint types, generated by Qwen2.5-72B-Instruct and programmatically verified against the IFEval evaluator such that every example passes `prompt_level_strict_acc`. These two datasets are complementary: Tulu handles the easy IFEval constraints the model already partially knows (JSON format, response language, postscript), while Argilla specifically targets the hard mechanical constraints where we measured high failure rates.

OpenCodeInstruct is filtered to examples with `average_test_score` ≥ 0.9, meaning only code solutions that pass at least 9 of 10 unit tests are included. This quality filter rejects noisy or incorrect code examples that would degrade training signal.

1.4 Data Mixing and Hyperparameters

Weighted sampling: math 2.0, IF (Tulu+Argilla) 4.0, code 2.0. Key optimizer settings: lr=5e-5 (initial), 3e-5 (continuation); warmup 200/100 steps; weight decay 0.01; grad clip norm 1.0; batch size 4; max length 1024. Weight decay and gradient clipping were added in v3 after observing noisy loss curves in earlier runs. Full hyperparameter table in supplementary.

1.5 Training Procedure

Two phases totalling 16,000+ steps. Phase 1: 8,000 steps from a fresh LoRA adapter, checkpointing every 1,000 steps via `save_state` + `save_weights_for_sampler`. Phase 2: resumed from step-8000 via `create_training_client_from_state`, lr reduced to 3e-5, fresh example ordering. All intermediate checkpoints evaluated on all three tasks.

2. Extensions

2.1 Programmatic Constraint Failure Analysis and Targeted Data Selection

After IFEval improvement plateaued despite scaling Tulu-3, we performed constraint-level failure analysis on Inspect AI `.eval` logs using `instruction_following_eval`, computing per-constraint pass/fail rates across all 541 prompts. Three clear failure clusters emerged:

- **Case (95–100%):** `english_lowercase` (100%), `english_capital` (92%), `capital_word_frequency` (84%)
- **Length/count (79–100%):** `nth_paragraph_first_word` (100%), `number_paragraphs` (93%), `number_words`/`number_sentences` (79%)
- **Keyword (79–84%):** `forbidden_words` (84%), `keywords:frequency` (81%), `keywords:existence` (79%)

Tulu-3 cannot address these failures: natural instruction following responses are not written to word-count specifications or written in all-lowercase. We identified `argilla/ifeval-like-data` (filtered): 56,339 examples generated by Qwen2.5-72B-Instruct and verified against the IFEval checker (`prompt_level_strict_acc = True`), covering all high-failure constraint types. Replacing Tulu-3 with Argilla in v4 produced an 18-point IFEval jump (41.9% → 60.2%), the largest single improvement of the project, validating that the bottleneck was data relevance, not capacity.

2.2 Manual GRPO Implementation on an SFT-Only API

GRPO generates G candidate responses, scores them with a reward function, and updates the model toward higher-reward outputs. IFEval is ideal for GRPO since the constraint checker provides a deterministic, programmatically computable reward. Tinker does not natively support RL, so we implemented GRPO from Tinker primitives. Each step: sample a prompt from argilla; generate G=16 responses via `create_sampling_client` at temperature 1.0 (with stop sequences to prevent spurious continuation turns); score each with `instruction_following_eval`.

Partial credit reward: $r = \frac{1}{K}\sum_{k=1}^{K} \mathbf{1}[\text{constraint}_k \text{ satisfied}]$, giving rewards in $[0, 1]$ (2/3 constraints → 0.667, not 0). Per-constraint kwargs passed individually to avoid collisions. Group-relative advantages: $A_i = (r_i - \bar{r})/(\sigma_r + \epsilon)$, Responses with $A_i > 0$ become weighted datums (token weights $\propto A_i$); $A_i \leq 0$ discarded (rejection sampling, simplified GRPO without reference model KL). Update: `forward_backward` + `optim_step`.

When all G samples receive identical rewards (variance = 0), no update occurs. At 60%+, this is common: easy constraints yield reward 1.0 for all samples; hard constraints yield 0.0 for all. No within-prompt variance ⇒ no gradient. This is GRPO's primary limitation at high performance and is discussed in Section 3.3. Training: 300–600 steps at lr=1e-5, starting from the v4b SFT checkpoint.

3. Results and Analysis

3.1 Experimental Progression

Version	IFEval	GSM8K	HumanEval	Key Change from Previous
Baseline (no FT)	22.8%	9.7%	0.6%	—
v1 (3B, 500 steps)	25.7%	26.0%	30.5%	Real data, 3B model
v1 (8B, 500 steps)	37.7%	51.3%	42.1%	Scaled to 8B
v2 (8B, 1500 steps)	41.9%	56.7%	46.3%	Orca-Math, 50k Tulu, CoT
v3 (8B, 2000 steps)	38.9%	58.9%	45.1%	System prompt — regression
v4 (8B, 2000 steps)	60.2%	61.1%	48.2%	Argilla replaces Tulu
v4b (8B, 2000 steps)	61.5%	63.8%	45.7%	Retrain with save_state fix
v4b + GRPO	61.3%	64.5%	46.3%	300-step GRPO on IFEval
v5 step 3000	62.5%	56.7%	42.7%	Large dataset, early checkpoint
v5 step 4000	60.6%	63.3%	43.3%	—
v5 step 5000	**63.5%**	63.7%	44.5%	IFEval peak
v5 step 8000	63.0%	**68.3%**	**45.7%**	GSM8K and HumanEval peak

3.2 What Worked and Why

Scaling to 8B. Moving from Llama-3.2-3B to Llama-3.1-8B produced the second-largest single improvement of the project (IFEval +12%, GSM8K +25.3%, HumanEval +11.6% in v1 comparisons). Larger models have more capacity

in their weight matrices to store task-specific patterns, and more robust representations learned during pretraining that fine-tuning can unlock. Notably, 3B and 8B results are not directly comparable as predictors of what works: the 3B model excelled on HumanEval early but struggled on GSM8K, while 8B showed the opposite pattern with better general alignment. This led to the decision to work exclusively with 8B for all subsequent experiments.

Argilla dataset selection. Replacing Tulu-3 with argilla in v4 produced an 18-point IFEval improvement — the largest single gain of the project. IFEval's hardest constraint types (lowercase, word count, forbidden words) require the model to monitor its own output during generation, a skill absent from natural instruction-following data. Argilla provides verified demonstrations of exactly these constraint types, with no degradation on GSM8K or HumanEval.

Multi-source math ensembling with CoT. Combining GSM8K, Orca-Math, and MetaMathQA with chain-of-thought prefixes improved GSM8K from 51.3% (v1) to 68.3% (v5). CoT prefixes teach the model to decompose problems before answering; since GSM8K scoring extracts the final number via regex, verbose intermediate reasoning does not penalize the score.

Extended training. The v5 loss curve declined monotonically across 8,000 steps with scores improving in tandem. At batch size 4 and 2,000 steps, only ∼2% of the 363k-example dataset is seen — far too little for convergence. Extending to 8,000 steps improved GSM8K by 7pp relative to v4b, and the curve showed no sign of overfitting.

Optimizer improvements. Weight decay (0.01) and gradient clipping (norm 1.0) added in v3 stabilized training and smoothed loss curves in subsequent runs. Their absence in v1–v2 likely caused the noisy, non-monotonic loss behavior observed in those runs.

3.3 Negative Results and Failed Experiments

System prompt on Tulu data (v3 regression). We prepended a constraint-awareness system prompt to all 80,000 Tulu examples, hypothesizing it would activate constraint-following behavior. IFEval instead regressed from 41.9% to 38.9%. Post-hoc constraint analysis confirmed the system prompt worsened every constraint category. Most likely cause: the model learned to prepend a constraint-checking preamble, causing failures on prompt-level strict scoring (which requires compliance from the first token). Additionally, Tulu examples without explicit constraints were now trained with a constraint-checking system prompt, creating incoherent signal.

Tulu-3 scaling (diminishing returns). Increasing Tulu from 50k to 80k with higher IF weight did not improve IFEval. Generic instruction following data has a ceiling for mechanical constraint types — the model learns to follow natural language instructions but Tulu does not teach word counting or comma avoidance, which require fundamentally different training signal.

GRPO reward sparsity. GRPO produced modest gains (GSM8K +0.7%, HumanEval +0.6%) but did not improve IFEval. At 60%+ accuracy, most prompts fall into degenerate cases: easy prompts where all G=16 samples score 1.0, or hard prompts where all score 0.0. Both produce zero variance and no gradient. Only prompts where the model is inconsistent produce non-zero advantage. A denser reward signal or targeted selection of high-failure prompts would be needed for GRPO to be effective at this performance level.

Early stopping trade-offs. No single checkpoint dominates all three tasks: IFEval peaks at step 5000 (63.5%) while GSM8K and HumanEval peak at step 8000 (68.3%, 45.7%). Step 8000 was selected for submission as it maximizes average score.

3.4 Leaderboard Comparison

Task	Our Best	Competition Leader	Gap
IFEval	63.5% (step 5000)	72.5%	−9.0%
GSM8K	68.3% (step 8000)	71.5%	−3.2%
HumanEval	48.2%	59.4%	−11.2%

Supplementary material (LLM Usage and Tinker Feedback) continues on the following page.

Supplementary Material

Full Hyperparameter Table

Parameter	Final Value	Rationale
Learning rate	5e-5 (initial), 3e-5 (continuation)	Reduced for continuation to avoid overshooting
Warmup steps	200 (initial), 100 (continuation)	Linear warmup stabilizes early rank-128 updates
Weight decay	0.01	L2 regularization; reduces overfitting on large dataset
Gradient clip norm	1.0	Prevents destabilizing spikes from hard batches
Adam β_1, β_2	0.9, 0.95	Standard values for LLM fine-tuning
Batch size	4	Tinker API constraint
Max sequence length	1024	Captures full chain-of-thought reasoning steps
LoRA rank	128	Multi-task adaptation capacity

Suggested Figures (placeholders for final submission)

The following figures are recommended to strengthen the visual presentation. Replace each `\fbox` with `\includegraphics` once the image files are prepared.

[Figure 1: Task accuracy across training versions]
Grouped bar chart: x-axis = version (v1–v5 step 8000), y-axis = accuracy, 3 bars per group.
`\includegraphics[width=\linewidth]{figures/version_comparison.png}`

Figure 1: IFEval, GSM8K, and HumanEval accuracy across all training versions.

[Figure 2: v5 training loss curve (steps 1,000–8,000)]
Line plot of 100-step rolling average loss. Data: step 1k=0.422, 3k=0.412, 5k=0.395, 8k=0.364.
`\includegraphics[width=\linewidth]{figures/loss_curve_v5.png}`

Figure 2: Training loss over 8,000 steps showing continued convergence throughout the run.

[Figure 3: IFEval constraint failure rates by type (v2 analysis)]
Horizontal bar chart sorted by failure rate. Top failures: `english_lowercase` (100%), `number_paragraphs` (100%), `number_highlighted_sections` (90%), `forbidden_words` (84%).
`\includegraphics[width=\linewidth]{figures/constraint_failures.png}`

Figure 3: Per-constraint-type failure rates from v2 evaluation logs, motivating the argilla dataset selection.

4. LLM Usage

Claude (Anthropic, `claude-sonnet-4-6`) was used extensively throughout this project. All usage is disclosed below.

Code writing. Claude wrote the majority of code in this project, including: all versions of `evaluation/train_and_publish.py` (dataset loaders, weighted sampler, training loop, checkpoint saving logic); `evaluation/grpo_train.py` (the full GRPO implementation); and analysis scripts including the constraint failure parser. Claude debugged Tinker API errors throughout, including identifying correct method signatures (`build_generation_prompt`, `save_state`, `SampledSequence.tokens`, `create_training_client_from_state`), resolving import errors, and fixing Windows-specific issues (SSH key setup, PowerShell venv activation, HuggingFace Hub symlink warnings).

Dataset research and selection. Claude searched HuggingFace for datasets matching the diagnosed failure modes and identified `argilla/ifeval-like-data`, `microsoft/orca-math-word-problems-200k`, and `meta-math/MetaMathQA`. Claude streamed first examples from each dataset to inspect schemas before writing

loading functions. The decision to use a verified constraint dataset rather than generating synthetic data was made by the students on ethical grounds (training on data engineered from the test constraint distribution felt too close to test set contamination).

Experimental design. Claude participated in strategic decisions including model size selection, data mixing ratios, learning rate scheduling, checkpoint interval selection, and GRPO algorithm design. Claude provided technical explanations of GRPO mechanics, LoRA theory, and reward signal design. Key decisions — the constraint failure analysis approach, the choice to pursue argilla over synthetic generation, the overnight training strategy, and the decision to implement GRPO despite Tinker's SFT-only API — were made by the students.

Analysis and interpretation. Claude helped interpret evaluation results and co-wrote the constraint failure analysis script. The insight that constraint failure rates should directly inform dataset selection (rather than prompt engineering or system prompts) emerged from discussion between the students and Claude.

Report writing. This report was drafted with Claude's assistance, based on a running `CHANGES.md` changelog maintained by the students throughout the project. All experimental results, failure analyses, and interpretations reflect actual outcomes.

5. Tinker Platform Feedback

What was the hardest part? The largest source of friction was the gap between Tinker's SFT-oriented API and the more advanced training paradigms the project documentation explicitly encourages. Implementing GRPO required significant exploratory debugging: discovering that `save_state` (not `save_weights`) is the correct method for producing a resumable training checkpoint; that sampler weights paths (`/sampler_weights/`) and training state paths (`/weights/`) use different formats and cannot be used interchangeably; and that `create_training_client_from_state` rejects sampler paths with an opaque 400 error. A taxonomy of checkpoint path formats in the documentation would have saved several hours of debugging. Windows compatibility was a secondary friction point — the project documentation and tinker-cookbook appear to assume a Unix environment, and several setup steps (SSH key configuration, HuggingFace Hub symlink behavior, venv activation syntax) required workarounds not covered in the materials.

Did you use the tinker-cookbook? Yes, extensively. The cookbook was indispensable for conversation rendering (`get_renderer`, `build_generation_prompt`), tokenization (`get_tokenizer`), and training datum construction (`conversation_to_datum`, `TrainOnWhat`). These abstractions work well and significantly reduce boilerplate. What was missing: (1) documentation of `build_generation_prompt`'s return type — it returns a `ModelInput` object directly rather than a token list, which is non-obvious and caused the GRPO sampling to fail initially; (2) examples of advanced workflows such as checkpoint resumption and multi-client setups (training client + sampling client simultaneously); (3) explicit documentation of which methods are available on `TrainingClient` vs `ServiceClient` without needing to use `dir()` and `inspect.signature()` to discover them.

What one thing would make Tinker significantly better? Native GRPO or PPO support via a `create_grpo_training_client` analogous to `create_lora_training_client`. The project documentation lists reinforcement learning as an encouraged extension, and Tinker already possesses all necessary primitives (sampling, gradient computation, checkpoint management). However, combining these into a correct RL loop requires substantial undocumented engineering, and the absence of a reference model for computing KL penalties means our implementation is a simplified approximation. A higher-level RL client that handles sampling, scoring, advantage computation, and reference model management internally would dramatically lower the barrier and produce more principled experiments.

Would you use Tinker again? Yes. The ability to train 8B models through a simple Python API without managing GPU infrastructure — no CUDA configuration, no cloud instance setup, no containerization — was genuinely valuable for an academic project where the goal is experimentation rather than infrastructure engineering. Training a 8B model for 8,000 steps with checkpointing completed reliably overnight. The platform is clearly well-suited to the SFT use case it is designed for. With improved documentation and RL support it would be a compelling platform for more advanced research workflows as well.